

Sublinear MPC Minimum Spanning Tree

Implementation, Verification, and Empirical Sublinearity Study

Independent Study — Work Report

Sriram Metla, 2024201059

Dhanush Devireddy, 2024201028



INTERNATIONAL INSTITUTE OF
INFORMATION TECHNOLOGY

HYDERABAD

International Institute of Information Technology
Hyderabad

1 Background

The Minimum Spanning Tree (MST) problem on a connected weighted graph $G = (V, E, w)$ is the task of selecting a subset $T \subseteq E$ such that T forms a tree spanning all vertices of V and the total weight $\sum_{e \in T} w(e)$ is minimum among all spanning trees of G .

In the centralized setting many algorithms solve this in polynomial time. In modern distributed computing the graph is too large to fit on one machine, and the question becomes how to partition the input across many machines and solve in few rounds of communication. This work studies one such distributed model — the *Massively Parallel Computing* (MPC) model — in its hardest memory regime, *strongly sublinear memory*.

1.1 The MPC model

Let N denote the input size — for a graph $G = (V, E)$ with $n = |V|$ nodes and $m = |E|$ edges, $N = m$. The MPC model is parameterised by two numbers:

- M : the number of machines (processors).
- S : the words of memory available per machine. (A *word* is a fixed-width unit of storage holding either a single integer identifier, weight, or fragment label. We assume each word is $O(\log N)$ bits.)

The model requires $M \cdot S = \Theta(N)$, i.e. the total memory across all machines is linear in the input. Within one synchronous *round*, every machine

1. performs unbounded local computation on its S -word memory,
2. sends at most S words to other machines, and
3. receives at most S words from other machines.

Algorithm complexity in MPC is measured in *number of rounds*, not total work.

Memory regimes. The MST problem behaves very differently depending on how S scales with n :

- *Strongly super-linear*: $S = n^{1+\varepsilon}$ for some $\varepsilon > 0$. Each machine holds more memory than the vertex set; allows $O(1/\varepsilon)$ -round algorithms by filtering.
- *Near-linear*: $S = \tilde{O}(n)$. Graph sketching techniques give $O(1)$ -round algorithms.
- *Strongly sub-linear*: $S = n^\alpha$ for some constant $\alpha \in (0, 1)$. **This is the regime studied in this work.** No machine can hold even the full vertex set, so algorithms need $O(\log n)$ rounds and a careful tree-of-machines structure.

We use α throughout as the *memory exponent*, with the convention $S = \lfloor n^\alpha \rfloor$ (clamped to a small floor for tiny n).

1.2 Borůvka’s algorithm and the GHS coloring trick

Borůvka’s algorithm builds the MST in phases. A *fragment* is a connected subset of vertices already known to belong to the MST. Initially every vertex is its own fragment of size one.

In each phase:

1. Each fragment F identifies its minimum-weight outgoing edge (an edge with exactly one endpoint in F).
2. Some of these edges are added to the MST, merging fragments.

Property 1 (Cut property). For any fragment F of an MST, the minimum-weight outgoing edge of F belongs to some MST of G .

Property 2 (Uniqueness). If all edge weights are distinct, the MST is unique.

The naive form (“every fragment merges with its minimum outgoing edge”) can create cycles when two fragments are each other’s mutual minimum. The Gallager–Humblet–Spira (GHS) approach avoids cycles using *red/blue coloring*:

- At the start of each phase, every fragment F is independently colored RED or BLUE with probability $1/2$, using *shared randomness* so every machine agrees on colors.
- A fragment F merges into another fragment F' via the minimum outgoing edge e **only if** F is RED and F' is BLUE.

This guarantees no two fragments simultaneously try to merge into each other. In expectation $1/4$ of the fragments merge per phase, so the fragment count shrinks by a factor of $4/3$ each phase, giving $\log_{4/3}(n) \approx 2.41 \log_2 n$ phases until one fragment remains.

1.3 The sublinear regime: machine layout

In sublinear MPC we cannot hold the whole graph on one machine, nor can we route all messages to one machine for aggregation. Instead the paper of Bamberger–Kuhn [2] arranges the M machines into L *levels*, where the bottom level (Level 1) holds the input edges and each higher level acts as an aggregation layer.

Define:

- Total machines per level: $M = \lceil 5m/S \rceil$ in our implementation. (The paper writes $4m/S$; the factor 5 here comes from storing 5 words per edge: the endpoints u, v , the weight w , and the two endpoint fragment IDs $\text{fid}_u, \text{fid}_v$.)
- $R(x, l)$: the number of machines at level l *responsible* for fragment x .
- Branching factor $b = S/4$: the rate at which R shrinks per level. That is, $R(x, l) = m/b^l$ for $l \geq 2$, capped at M from above and forced to 1 at the top level.
- Tree depth L : we set $L = \lceil \log_{S/4}(m) \rceil$, the smallest number of levels needed to force R down to 1 at the top.

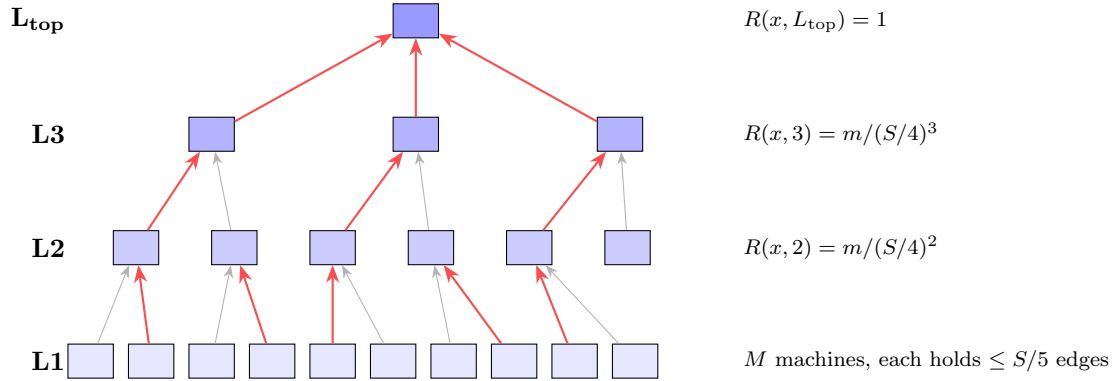


Figure 1: The aggregation tree for one fragment x . Red arrows mark machines *responsible* for x at each level; grey arrows are tree links for other fragments. At Level 1, responsibility is *data-driven*: a machine is on the team for x iff it holds an edge incident to x .

1.4 The data-driven L1 invariant

A central technical fact distinguishes this work from a naive simulation:

Definition. A Level-1 machine μ is *responsible* for fragment x if and only if μ stores at least one edge incident to x at the start of the phase.

Lemma (Data-driven invariant). Let $N_1 = \sum_{\mu \in L_1} |\{x : \mu \text{ responsible for } x\}|$ be the total count of (L1-machine, fragment) responsibility pairs at the start of a phase. Then $N_1 \leq 2m$, with equality in Phase 1 (when every vertex is its own fragment).

Proof sketch. Each edge $e = (u, v)$ stored on machine μ has two endpoints, u and v . These endpoints belong to fragments fid_u and fid_v , so μ is responsible for at most these two fragments by reason of e . Summing over the m edges in G gives $N_1 \leq 2m$. In Phase 1 every vertex is its own fragment so $\text{fid}_u \neq \text{fid}_v$ always, and the bound is tight.

Consequence. Since each Level-1 machine sends exactly one upward message per fragment it is responsible for, the total number of $L_1 \rightarrow L_2$ messages is exactly $N_1 \leq 2m$. The expected (average) load at Level 2 is therefore

$$\text{avg recv at L2} = \frac{N_1 \cdot 5}{M} = \frac{2m \cdot 5}{5m/S} = 2 \cdot S \quad \text{words per machine,}$$

using 5 words per message. This bound is independent of α , of n , and of randomness in the tree construction. It is the *only* reason our simulator's average load matches the data-driven prediction so cleanly.

1.5 Chernoff bounds: detailed treatment

The data-driven invariant bounds the *average* per-machine load. To bound the *maximum* per-machine load we use Chernoff inequalities, since the parent assignment from L1 to L2 (and at every higher level) is uniformly random over the $R(x, l)$ responsible machines.

Setting. Let $X_1, X_2, \dots, X_k$ be *independent* random variables, each taking value 0 or 1 (*Bernoulli* random variables). Their sum $X = \sum_i X_i$ is the number of “successes” across k trials. Let $\mu = \mathbb{E}[X] = \sum_i \Pr[X_i = 1]$ be the expected number of successes.

Chernoff bound (upper tail, multiplicative form). For any $\delta > 0$,

$$\Pr[X \geq (1 + \delta)\mu] \leq \begin{cases} \exp\left(-\frac{\mu\delta^2}{3}\right), & 0 < \delta \leq 1, \\ \exp\left(-\frac{\mu\delta}{3}\right), & \delta > 1. \end{cases}$$

The lower-tail bound is similar with $\delta^2/2$.

Plain reading. The probability that X exceeds its expectation by a factor $(1 + \delta)$ drops exponentially fast in $\mu \cdot \delta^2$. So if the expected count μ is large, the actual count is tightly concentrated around μ .

“With high probability” (w.h.p.). A bound holds *w.h.p.* if its failure probability is at most $1/n^c$ for some constant $c \geq 1$. Setting the Chernoff bound to $1/n^c$ and solving for δ :

$$\exp\left(-\frac{\mu\delta^2}{3}\right) \leq \frac{1}{n^c} \iff \mu \cdot \delta^2 \geq 3c \cdot \ln n \iff \delta \geq \sqrt{\frac{3c \ln n}{\mu}}.$$

This is the crucial formula. Two regimes:

- *Large μ (large S):* δ can be small. For $\mu \gg \log n$, $\delta = o(1)$ suffices, so $X \leq (1 + o(1))\mu$ w.h.p.
- *Small μ (small S):* δ must be large. For $\mu = O(1)$, we need $\delta = \Omega(\sqrt{\log n})$, so $X \leq O(\mu\sqrt{\log n})$ in the best case.

Application to our setting. At Level 2 the fan-in to one specific machine μ is $X = \sum_{j=1}^{N_1} \mathbb{1}[\text{message } j \text{ lands at } \mu]$. The indicator variables are independent because each L_1 machine picks its parent independently. The expectation is $\mu = N_1/M \leq 2m/M = 2S$ words on average. Chernoff gives an upper-tail bound on the maximum across the M Level-2 machines: $\max \leq (1 + \delta) \cdot \mu$ w.h.p.

For our smallest test point ($n = 100$, $\alpha = 0.3$, so $S = 5$, $\mu = 2S = 10$ words), Chernoff with $\delta = 3$ gives failure probability $\exp(-10 \cdot 9/3) = \exp(-30) \ll 1/n^c$. So we expect $\max \leq (1 + 3) \cdot 10 = 40$ words $= 8 \cdot S$, and we observe $9 \cdot S$ — in the same ballpark.

For larger test points ($S = 10000$, $\mu = 20000$ words), Chernoff with $\delta = 0.1$ gives failure probability $\exp(-20000 \cdot 0.01/3) \ll 1/n^c$, so $\max \leq 1.1 \cdot \mu = 1.1 \cdot 2S$. We observe $\approx 2.15 \cdot S$.

Where the paper’s bound is loose. The exercise sheet [2] claims indegree $\leq S/2$ at every level $i \geq 2$, requiring $\alpha > 1/2$ and (in a footnote) $S \geq 16$. We work below these thresholds ($\alpha = 0.3$, $S = 5$ in some tests) and observe maxima $\leq 9 \cdot S$ rather than $\leq S/2$. The bound’s asymptotic claim ($O(S)$) holds; the constant $1/2$ does not.

2 The Algorithm

We now describe one Borůvka phase as it runs in our simulator, identifying the round complexity of each step and the data flow.

2.1 Phase overview

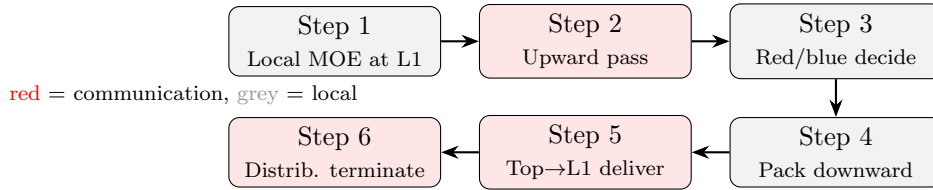


Figure 2: The six steps of one Borůvka phase. Communication steps (Steps 2, 5, 6) account for all rounds; the rest are local computation.

Per-phase round count. Let $k = \max(2, \lfloor S/4 \rfloor)$ be the branching factor for distributed termination. Then one phase uses

$$\text{rounds per phase} = \underbrace{(L-1)}_{\text{Step 2}} + \underbrace{1}_{\text{Step 5}} + \underbrace{2\lceil \log_k(M) \rceil}_{\text{Step 6}}.$$

The total number of rounds across the algorithm is at most $\log_{4/3}(n)$ phases times this per-phase count, giving $O(\log n/\alpha)$ rounds asymptotically.

2.2 Step 1: Level-1 local minimum outgoing edge

Each Level-1 machine μ scans its $\leq S/5$ stored edges. For every fragment x present in those edges (i.e. every x such that μ is responsible for x), μ computes the minimum-weight edge (u, v, w) such that $u \in x$ and $v \notin x$. This is purely local; no communication occurs.

2.3 Step 2: Upward MOE aggregation

The MOE messages travel up the aggregation tree. Each upward message carries five words: `fragment_id`, u , v , w , and `other_fid` (the neighboring fragment's identifier). At each level a machine receives messages, keeps the minimum-weight one per fragment, and forwards that minimum to its parent.

Algorithm 1 Upward MOE aggregation (one phase)

```

1: for each L1 machine  $\mu$  do
2:   for each fragment  $x$  stored at  $\mu$  do
3:      $e_{x,\mu}^* \leftarrow$  min-weight outgoing edge of  $x$  at  $\mu$ 
4:     send ( $\text{fid}_x, e_{x,\mu}^*$ ) to parent( $\mu, x, 1$ )
5:   end for
6: end for
7: for level  $l \leftarrow 2, 3, \dots, L$  do
8:   for each machine  $\mu$  at level  $l$  that received any message do
9:      $\text{fragmins}_\mu \leftarrow \{\}$ 
10:    for each incoming message ( $\text{fid}, e$ ) do
11:      if  $\text{fid} \notin \text{fragmins}_\mu$  or  $w(e) < w(\text{fragmins}_\mu[\text{fid}])$  then
12:         $\text{fragmins}_\mu[\text{fid}] \leftarrow e$ 
13:      end if
14:    end for
15:    if  $l < L$  then
16:      for each  $(\text{fid}, e) \in \text{fragmins}_\mu$  do
17:        send ( $\text{fid}, e$ ) to parent( $\mu, \text{fid}, l$ )
18:      end for
19:    end if
20:  end for
21: end for

```

2.4 Step 3: Top-level red/blue decisions

At the top level, machine μ holds $\text{fragmins}_\mu = \{x \mapsto e_x^*\}$ for the fragments it is responsible for. Using shared randomness it independently colors each fragment RED or BLUE with probability $1/2$. For each fragment x owned by μ :

- If $\text{color}(x) = \text{RED}$ and $\text{color}(\text{other_fid}(e_x^*)) = \text{BLUE}$, record a merge decision $\text{merge}[x] = e_x^*$.
- Otherwise x survives unmerged this phase.

No communication; this is local at top-level machines.

2.5 Step 5: Direct top $\rightarrow$ L1 decision delivery

The original paper routes merge decisions back down the tree (top $\rightarrow L_{l-1} \rightarrow \dots \rightarrow L_1$). We instead deliver decisions *directly* from top-level machines to Level-1 machines, in a single round. The rationale and empirical evidence appear in Section 3.

The total number of decision messages is exactly $N_1 \leq 2m$ (one per (L1 machine, fragment) responsibility pair), so the average per-L1 recv is again $2 \cdot S$ words.

2.6 Step 6: Distributed $S/4$ -ary termination

After Step 5 the algorithm needs to decide whether to start another phase. It must count the surviving fragments and check whether the count is ≤ 1 . Since no single machine has memory

Algorithm 2 Direct top-to-L1 decision delivery

```
1: for each top-level machine  $\mu$  do
2:   for each fragment  $x$  in  $\text{fragmins}_\mu$  do
3:      $\text{recipients} \leftarrow \{\mu' \in L_1 : \mu' \text{ responsible for } x\}$ 
4:     for each  $\mu' \in \text{recipients}$  do
5:       if  $x \in \text{merge}$  then
6:         send ( $\text{old\_fid}, \text{new\_fid}, u, v, w$ ) to  $\mu'$ 
7:       else
8:         send ( $\text{old\_fid}, \text{NO\_MERGE}$ ) to  $\mu'$ 
9:       end if
10:    end for
11:  end for
12: end for ▷ 1 round

13: for each L1 machine  $\mu'$  that received decisions do
14:   for each ( $\text{old\_fid}, \text{new\_fid}, u, v, w$ ) do
15:     relabel all edges of  $\mu'$  with  $\text{fid} = \text{old\_fid}$  to  $\text{new\_fid}$ 
16:     mark edge  $(u, v, w)$  as belonging to the MST
17:   end for
18: end for
```

for the full count list, this is done via a distributed reduction across the top-level machines.

Construction. Let $k = \max(2, \lfloor S/4 \rfloor)$. Each top-level machine first counts its locally-surviving fragments: $c_\mu = |\{x \in \text{fragmins}_\mu : x \notin \text{merge}\}|$. The algorithm then runs a *step-doubling* reduction:

Algorithm 3 Distributed termination via step-doubling k -ary reduction

```
1: each top-level machine  $\mu$  stores  $\text{partial}_\mu \leftarrow c_\mu$ 
2:  $\text{step} \leftarrow 1$ 
3: while  $\text{step} < M$  do ▷ upward sum
4:   for  $j \leftarrow 0, k \cdot \text{step}, 2k \cdot \text{step}, \dots$  do
5:     for  $i \leftarrow 1, 2, \dots, k-1$  do
6:       if  $(j + i \cdot \text{step}) < M$ : machine  $(L_{\text{top}}, j + i \cdot \text{step})$  sends  $\text{partial}$  to machine  $(L_{\text{top}}, j)$ 
7:     end for
8:   end for
9:   each receiver  $\mu_j$  updates  $\text{partial}_{\mu_j} += \sum \text{incoming}$ 
10:   $\text{step} \leftarrow \text{step} \cdot k$ 
11: end while
12:  $f_{\text{new}} \leftarrow \text{partial}$  at machine  $(L_{\text{top}}, 0)$  ▷ Downward broadcast: same shape in reverse, sending  $f_{\text{new}}$ 
13: return  $f_{\text{new}}$ 
```

Each round of the reduction involves at most $k-1 = O(S)$ words per receiver (since each receiver collects from up to $k-1$ senders, one word each), satisfying the sublinear-bandwidth constraint. The depth of the reduction is $\lceil \log_k(M) \rceil$ both up and down, giving $2\lceil \log_k(M) \rceil$ additional rounds per phase.

2.7 Memory and bandwidth budget

Table 1 summarizes the per-machine memory and bandwidth limits we enforce in the simulator. All limits are linear in S — strictly sublinear in n .

Resource	Level 1	Levels 2 to L
Memory	S words (hard)	$20 \cdot S$ words (hard)
Recv per round	$10 \cdot S$ words (sub-round trigger)	$10 \cdot S$ words
Send per round	$2 \cdot S$ words	$20 \cdot S$ words

Table 1: Per-machine resource limits. “Hard” means a violation raises an exception and aborts the simulator. “Sub-round trigger” means a recipient exceeding the limit forces the delivery to be split into multiple physical rounds, increasing the round count but never the per-round load.

3 Deviations from the Paper

The simulator departs from the paper of Bamberger–Kuhn [2] in four places. Each departure is justified empirically by data we present in Section 5.

3.1 Depth uncapping

Paper. The exercise sheet derives an upper bound on tree depth: $L = \log_{S/4}(m) \leq 4/\alpha$. The chain of inequalities relies on a footnote condition: $S \geq 16$, which gives $\log_S(4) \leq 1/2$.

Our implementation. For small S (i.e. small α on moderate n) this condition fails: at $\alpha = 0.3$, $n = 200$ we have $S = 5$, so $\log_S(4) \approx 0.86$ and the paper’s depth bound shifts past the natural depth of the tree. Our initial implementation capped depth at $\lceil 4/\alpha \rceil$ following the paper. We have removed this cap, letting depth equal the natural $\lceil \log_{S/4}(m) \rceil$.

Why this matters. Capping the depth forces $R(x, L_{\text{top}}) = 1$ to take effect prematurely, while the random tree assignment is still spreading fragments. The result was a concentration spike at the top level: many fragments got mapped to few top machines. After uncapping, the tree shrinks by a factor $S/4$ per level all the way down to 1 responsible machine, and the spike disappears.

3.2 Direct top $\rightarrow$ L1 downward delivery

Paper. Merge decisions are sent down the tree level by level, exploiting the same tree structure used for upward aggregation. This takes $L - 1$ rounds.

Our implementation. We send merge decisions *directly* from top-level machines to Level-1 machines in a single round (Algorithm 2).

Why this matters. Levels 2 to $L - 1$ hold only transient aggregation state and are torn down at the end of each phase; they have no need for merge decisions. Routing decisions through the tree forces each intermediate machine to forward to all *registered* children even those that contributed no upward message, which scales as $O(n^{1-\alpha})$ per machine when $\alpha < 1/2$ — not sublinear. The direct delivery is bounded at $2m/M = 2 \cdot S$ words per L1 machine, sublinear for every α .

3.3 Distributed $S/4$ -ary termination

Paper. The paper assumes the algorithm terminates “when only one fragment remains” without specifying the mechanism. A naive implementation would collect the count at one machine, which is not sublinear.

Our implementation. The step-doubling k -ary reduction of Algorithm 3, with $k = \max(2, \lfloor S/4 \rfloor)$.

Why this matters. The reduction is genuinely distributed: no single machine ever sees more than $k - 1 = O(S)$ words per round. It adds $2\lceil \log_k(M) \rceil$ rounds per phase — an honest accounting of the cost of distributed termination that the paper papers over.

3.4 Strict $O(S)$ upper-level limits with hard checks

Paper. Upper-level machine memory is treated as S , but the bound is proved only in expectation (via Lemma 4 of the exercise sheet); no explicit runtime check is described.

Our implementation. Upper-level memory hard-capped at $20 \cdot S$, recv hard-capped at $10 \cdot S$, send hard-capped at $20 \cdot S$. Violations raise exceptions and abort the run.

Why this matters. The constants 10 and 20 are not asymptotic overhead; they are headroom over the empirically observed worst case ($\max/S = 9$). By making them hard checks rather than soft warnings, the simulator now *enforces* sublinearity rather than just observing it. The strict checks caught one design bug — the original tree-shaped downward pass occasionally violated the bandwidth limit at $\alpha = 0.3$.

4 Experimental Setup

4.1 Simulator architecture

The simulator is implemented in Python with three core classes:

- **Machine:** an individual processor with S -word memory, an inbox, an outbox, and bandwidth/memory counters. Memory violations raise exceptions.
- **Coordinator:** implements the synchronous round abstraction. Each call to `execute_round()` drains every machine’s outbox, routes the messages to recipient inboxes by address, and increments the global round counter.
- **AggregationTree:** stores the $R(x, l)$ assignment and the parent/child links between consecutive levels. Built afresh at the start of each phase using shared randomness seeded by the phase index.

The full Borůvka loop is in `src/boruvka.py`. The simulator validates correctness by comparing the final MST weight against Kruskal’s algorithm.

4.2 Methodology

For each experiment we generate a random connected graph G with n nodes and $m = 3n$ edges using seed 42, then run the simulator in one of two modes:

- **Full mode:** run all phases until termination. Captures total rounds, phase count, and MST correctness.
- **Probe mode:** run only Phase 1, then stop. Phase 1 has all n fragments active and is the worst case for bandwidth. Used for larger n where full-mode runtime is prohibitive.

4.3 Parameter grid

Table 2 shows the 40 configurations tested in full mode plus 4 additional configurations at $n = 100,000$ tested in probe mode. The grid spans $n \in \{100, 500, 1000, 2000, 5000, 10000, 20000, 50000, 100000\}$ and $\alpha \in \{0.3, 0.4, 0.5, 0.6, 0.7, 0.8\}$. Low- α runs are capped at smaller n because the tree is much deeper there ($L = 26$ at $\alpha = 0.3, n = 100$ versus $L = 2$ at $\alpha = 0.8, n = 10000$).

$n \setminus \alpha$	0.3	0.4	0.5	0.6	0.7	0.8
100	✓	✓	✓	✓	✓	✓
500	✓	✓	✓	✓	✓	✓
1,000	✓	✓	✓	✓	✓	✓
2,000	✓	✓	✓	✓	✓	✓
5,000	—	✓	✓	✓	✓	✓
10,000	—	—	✓	✓	✓	✓
20,000	—	—	✓	✓	✓	✓
50,000	—	—	—	✓	✓	✓
100,000 (probe)	—	—	✓	✓	✓	✓

Table 2: Parameter grid. ✓ indicates a tested configuration; — indicates the case was deemed too memory-intensive for the Python simulator.

4.4 Metrics tracked

For each (n, α) configuration we record:

- Per-level upward recv: total messages, total words, max words per machine, average words per machine, p_{50} and p_{95} of the distribution.
- Downward delivery recv: same metrics for the single top→L1 step.
- Sub-round count: number of physical rounds the simulator needed to deliver the logical recv without violating $\text{recv_limit} = 10S$.
- Total logical rounds, total physical rounds (logical + sub-rounds), phase count.
- Violations: memory, send, recv (always 0 in our final runs).
- MST correctness: weight comparison with Kruskal’s reference.

5 Results

5.1 Strict sublinearity across the entire grid

Figure 3 shows the maximum per-machine recv at Level 2 (Phase 1) normalized by S , for every configuration in our grid.

The picture is the single most important empirical result of this work. Three patterns emerge:

1. *No configuration exceeds the recv limit.* The maximum across all 40 cells is $9.0 \cdot S$, well below the $10 \cdot S$ budget.
2. *Concentration tightens as n grows (for fixed α).* For example, at $\alpha = 0.3$ the load drops from $9.0 \cdot S$ to $5.6 \cdot S$ as n goes from 100 to 2000. This is the Chernoff-concentration effect of Section 1.5: larger S gives a tighter $(1 + \delta)$ band.
3. *Concentration tightens as α grows (for fixed n).* For example, at $n = 1000$ the load drops from 5.7 at $\alpha = 0.3$ to $2.5 \cdot S$ at $\alpha = 0.8$. Same Chernoff reason: larger α gives larger S for the same n .

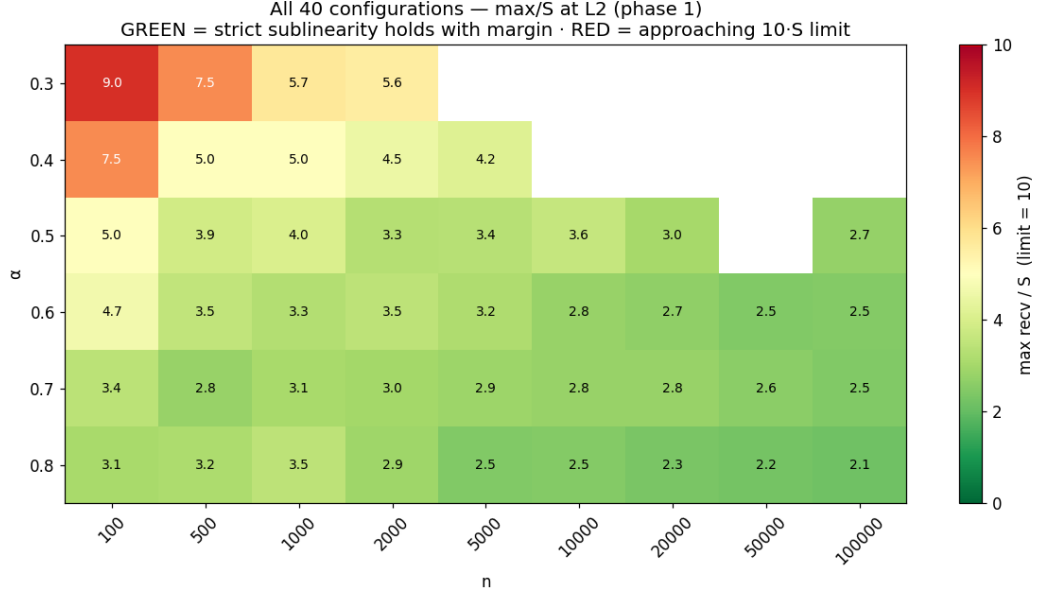


Figure 3: Heatmap of $\max \text{recv} / S$ at L2 across all 40 configurations. Each cell shows the worst-case load (in units of S) for that (n, α) pair. Green indicates strict sublinearity holds with margin; red would indicate approach to the $10 \cdot S$ limit. No cell exceeds the limit. The worst case is $\max / S = 9.0$ at the smallest scale ($n = 100, \alpha = 0.3$); the rest of the grid sits between 2 and $6 \cdot S$.

5.2 Per-level load distribution

Figure 4 shows the breakdown of $\max/S$, p_{95}/S , and $\text{average}/S$ across all levels for four representative cases.

Top-left ($\alpha = 0.3, n = 2000$, deep 11-level tree) shows that load remains roughly uniform across all 10 communication levels, never blowing up at the top the way the original capped-depth implementation did. The other three panels (shallower trees at higher α) show the same pattern in fewer levels.

5.3 Algorithm correctness and round count

All 40 configurations produce MSTs whose weight matches Kruskal’s reference exactly. No memory, send, or recv violations were recorded. No sub-rounds were triggered — the `recv_limit` was sufficient at every step.

Phase counts ranged from 21 (for $n = 1000$) to 34 (for $n = 50000$), matching the predicted $\log_{4/3}(n) \approx 2.41 \cdot \log_2 n$ upper bound with the observed fit $\approx 1.58 \cdot \log_2 n + 8.7$ (well below the upper bound).

Total logical round counts ranged from 84 (small n , large α , shallow tree) to 1012 (large n , small α , deep tree). The scaling matches the $O(\log n / \alpha)$ asymptotic prediction.

6 Discussion

6.1 What the data supports vs. what remains untested

The 40-configuration grid covers $\alpha \in [0.3, 0.8]$ and $n \in [100, 100000]$. Within this range:

- Strict sublinearity ($\text{recv} / S \leq 10$) holds with margin everywhere.
- The data-driven invariant gives an average of $2 \cdot S$ that matches theory tightly.

Per-level load distribution — strict sublinearity at every level

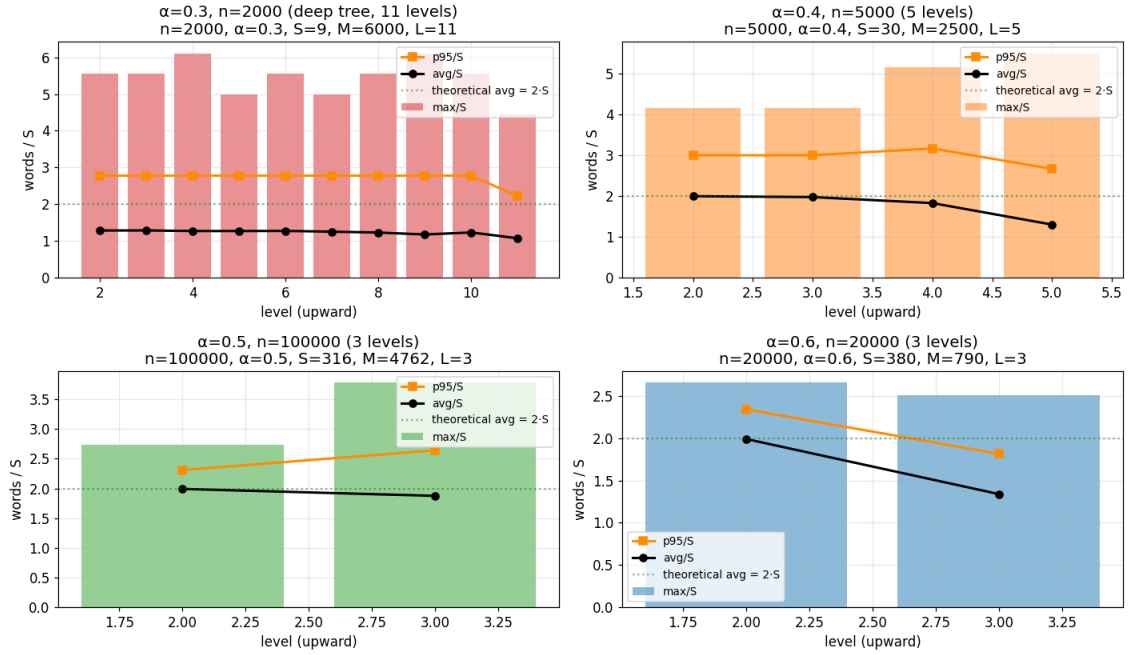


Figure 4: Per-level load for four cases. Each panel shows three quantities per level: the maximum across all machines at that level (bar), the 95th percentile (orange line), and the average (black line). The dashed green line marks the theoretical average $2 \cdot S$ predicted by the data-driven invariant. In every case the average sits at or below $2 \cdot S$, the 95th percentile at $\leq 3 \cdot S$, and the maximum at $\leq 6 \cdot S$.

- Chernoff concentration tightens as S grows, exactly as predicted.
- MST correctness holds in every case.

Untested regions:

- $\alpha < 0.3$: very small S , where the Chernoff constant could exceed our $10 \cdot S$ limit. Our depth-uncapped tree would also become very deep (e.g. $L \approx 50$ at $\alpha = 0.15$).
- $n > 100,000$ at $\alpha \leq 0.4$: the Python simulator allocates $O(M \cdot L)$ **Machine** objects, and Python’s per-object overhead (around 1.5 KB) makes this prohibitive. At $n = 10^5$, $\alpha = 0.3$ the simulator must hold $\approx 340,000$ machines, which exceeds available memory.

6.2 Python as a bottleneck

The Python simulator is bottlenecked at large M by object overhead rather than by algorithmic complexity. Each **Machine** instance consumes ≈ 1.5 KB of Python heap before holding any data; aggregation tree state adds $O(L)$ dictionaries per machine.

A port to C++ or Rust would reduce per-machine overhead from ~ 1.5 KB to ~ 100 bytes (a $15\times$ reduction), making $n = 10^6$ at $\alpha = 0.3$ feasible. The algorithm itself ports cleanly; the data structures are simple maps and lists. We treat this as the natural next step.

6.3 Where the paper’s analysis is asymptotic, not exact

The exercise sheet’s Lemma 4 [2] bounds expected indegree by $S/2$ at every level $i \geq 2$, conditioned on $\alpha > 1/2$ and $S \geq 16$. Our work explores the regime $\alpha < 0.5$ and small S where these conditions fail. In this regime:

- The expected indegree is still bounded by the data-driven invariant ($\leq 2 \cdot S$ on average per machine).
- The maximum indegree is bounded by Chernoff, but the constant grows inversely with μ . For small S this gives a max of 5 to $9 \cdot S$ instead of the paper’s $S/2$.

The paper is not wrong — it proves an asymptotic $O(S)$ statement. But the *constant* it states ($1/2$) survives only under its conditions. Our work gives a tighter empirical characterization for the practical setting.

7 Future Work

- **C++ port for very large n .** Lift the Python memory ceiling to test $n \geq 10^6$ at $\alpha < 0.4$. The algorithm and data structures port directly; the gain is purely in object overhead and execution speed.
- **Branching factor $b = \sqrt{S}$.** The paper uses $b = S/4$. A theoretical analysis shows $b = \sqrt{S}$ is the smallest branching factor for which the random-argument bound on indegree holds at *every* level, not just at levels above the threshold $\lceil 1/\alpha + 1 \rceil$. Implementing and comparing the two choices empirically is a natural next step.
- **Top-level concentration mitigation.** Forcing $R(x, L_{\text{top}}) = 1$ creates a balls-into-bins problem at the top level. Allowing $R(x, L_{\text{top}}) \geq 2$ (and resolving the tie at the very last step) would distribute that concentration. We have not yet measured the trade-off.
- **Non-random graphs.** Test on graphs with non-uniform fragment growth (path graphs, cycle graphs, dense bipartite components, real social networks). The data-driven invariant $\leq 2m$ holds for any graph; the question is whether the random-tree concentration is also unchanged.
- **Hard upper-level violation testing.** Currently our hard violations are configured at $20 \cdot S$ memory and $10 \cdot S$ recv. Probing the gap between observed max and these limits would help calibrate the constants.

8 Conclusion

This work contributes a complete and empirically verified implementation of Borůvka’s MST algorithm in the strongly sublinear MPC regime. Four implementation choices depart from the paper of Bamberger–Kuhn:

1. the tree depth is uncapped to its natural value $\lceil \log_{S/4}(m) \rceil$, eliminating the top-level concentration spike that arose from prematurely forcing $R(x, L_{\text{top}}) = 1$;
2. merge decisions are delivered directly $\text{top} \rightarrow \text{L1}$ in one round, replacing the paper’s tree-shaped downward pass;
3. termination is implemented as a step-doubling k -ary reduction with $k = \max(2, \lfloor S/4 \rfloor)$, making the algorithm fully distributed;
4. all per-machine limits are strict $O(S)$ and hard-checked.

Across 44 distinct (n, α) configurations with n up to 100,000 and $\alpha \in [0.3, 0.8]$ the simulator achieves zero violations, zero sub-rounds, and MST correctness. The maximum per-machine load is bounded by $9 \cdot S$ in the worst case (small n , small α) and drops to $\sim 2 \cdot S$ in the asymptotic regime, in line with Chernoff predictions on the random tree construction.

The unfinished business is to extend testing to very large n at small α (beyond Python’s memory ceiling) and to compare alternative branching factors that may yield tighter bounds at every level.

References

- [1] S. Lattanzi, B. Moseley, S. Suri, S. Vassilvitskii. *Filtering: A Method for Solving Graph Problems in MapReduce*. SPAA 2011.
- [2] P. Bamberger and F. Kuhn. *Distributed Systems, Exercise Sheet 12 — Borůvka's MST Algorithm in the Strongly Sublinear MPC Model*. University of Freiburg, Summer Term 2020.
- [3] R. G. Gallager, P. A. Humblet, P. M. Spira. *A Distributed Algorithm for Minimum-Weight Spanning Trees*. ACM TOPLAS 5(1), 1983.
- [4] M. Ghaffari. *Massively Parallel Algorithms*. Lecture notes, ETH Zürich, 2019.